

Regulamento Oficial: Hackathon “Operation: A Cobra Vai Fumar”

UNIMETROCAMP Wyden - Maratona de Programação e Criptografia

1. Introdução e Visão Geral

O Hackathon “Operation: A Cobra Vai Fumar” é uma maratona de desenvolvimento de software e resolução de problemas técnicos focada em **Criptografia e Engenharia de Software**. O evento é destinado exclusivamente aos estudantes de Computação da **UNIMETROCAMP Wyden**, visando integrar calouros e veteranos em um ambiente que simula os desafios reais do mercado de tecnologia.

O objetivo central é a aplicação prática de conceitos acadêmicos em um cenário de alta pressão, estimulando o pensamento crítico, a colaboração em equipe e a entrega de produtos mínimos viáveis (MVPs) funcionais.

2. Estrutura e Formação de Equipes

Para fomentar a troca de conhecimento e a mentoria interna, a composição das equipes deve seguir critérios rigorosos:

- **Composição:** Cada equipe deve ter entre **3 e 6 participantes**.
 - **Diversidade Técnica:** É obrigatória a presença de, no mínimo, **1 Calouro** (1º ano) e **1 Veterano** (anos avançados).
 - **Identificação:** As equipes devem operar sob um **Codiname**, reforçando a temática da operação.
 - **Estabilidade:** Não serão permitidas alterações na composição das equipes após o início oficial do cronômetro.
-

3. Configuração da VM e Infraestrutura Técnica

O ambiente de desenvolvimento será padronizado para garantir que o foco seja exclusivamente na lógica e implementação.

3.1. Tabela de Tecnologias e Dependências

Categoria	Tecnologia	Finalidade	Observações
Ambiente Base	Ubuntu 22.04 LTS / Windows (WSL2)	Sistema Operacional	Estabilidade e suporte a ferramentas POSIX.
IDE Principal	Visual Studio Code (VS Code)	Desenvolvimento	Pré-configurado com extensões essenciais: Python, ESLint, Prettier, Debugger for Java.
Linguagem: Python	Python 3.10+	Lógica de Missão	Bibliotecas: cryptography , numpy , flask ou fastapi (para interface web).
Linguagem: JS/TS	Node.js 18+	Interface/Backend	Frameworks: React , Next.js , Express . Dependências: axios , crypto-js .
Linguagem: Java	OpenJDK 17	Lógica de Missão	Ferramentas: Maven ou Gradle . Bibliotecas: Spring Boot , JCA .
Linguagem: C/C++	GCC/G++ 11	Performance	Bibliotecas: OpenSSL , Math.h .
Versionamento	Git 2.34+	Gestão de Código	Configurado com SSH/PAT para o GitHub.
Banco de Dados	SQLite / Redis	Persistência Local	Útil para armazenar chaves e logs de missões.
IA e Auxiliares	Copilot, ChatGPT, Templates	Produtividade	Permitidos, desde que o código final seja autoral.

4. Fluxo dos Participantes (Input → Processamento → Output)

O hackathon é estruturado como um pipeline de processamento de dados e desafios:

1. Input (Entrada):

- As equipes recebem um arquivo criptografado via QR Code.
- **Desafio Zero:** Resolução de um enigma lógico para obtenção da senha de extração. O enigma é: “Um fenômeno auto-replicante de baixa entropia que gera uma força de ligação chamada compaixão. O que eu sou?”
- Recebimento dos briefings técnicos das 3 Missões (RSA, Vigenère e Escada de Assalto).

2. Processamento (Desenvolvimento):

- **Análise:** Interpretação dos requisitos matemáticos e lógicos.
- **Codificação:** Implementação do algoritmo central (Core Logic).
- **Interface:** Desenvolvimento de uma UI funcional para interação com o usuário.
- **Isolamento:** Cada missão deve ser desenvolvida de forma independente, sem reaproveitamento de código entre elas.

3. Output (Entrega):

- Código-fonte versionado no GitHub (uma branch por missão).
- Documentação técnica (README.md) clara e reproduzível.
- **Apresentação Final:** Demonstração ao vivo (Live Demo) iniciando do `git clone` até a execução.

5. Detalhamento das Missões (Briefing Técnico)

Baseado no sistema oficial da operação, as equipes devem completar três missões sequenciais.

Missão 1: RSA Simplificado (Criptografia Assimétrica)

- **Objetivo:** Implementar o algoritmo RSA SIMPLIFICADO para criptografar e descriptografar mensagens.
- **Input:**
 - Mensagem (M): string de texto.
 - Primo p: número primo.

- Primo q : número primo.
- Chave pública e : número inteiro coprimo com ϕ .
- Modo: “criptografar” ou “descriptografar”.
- **Processamento:**
 - Conversão de texto para números usando Tabela ASCII (A=65, B=66, etc.).
 - Manter espaços em branco no texto.
 - Tratar letras maiúsculas e minúsculas corretamente.
 - Validação de que p e q são números primos.
 - Validação de que e e ϕ são coprimos ($\text{MDC} = 1$).
 - Implementar tratamento de erros para entradas inválidas.
 - Cálculo de $n = p * q$ e $\phi = (p-1) * (q-1)$.
 - Cálculo da chave privada d .
- **Output:** Texto cifrado (numérico) ou mensagem original, com exibição dos cálculos intermediários de n , ϕ e d .

Missão 2: Cifra de Vigenère (Criptografia Polialfabética Clássica)

- **Objetivo:** Implementar a CÍFRA DE VIGENÈRE para criptografar e descriptografar mensagens.
- **Input:**
 - Mensagem (M): string de texto (letras e espaços).
 - Chave (K): string apenas com letras (sem espaços).
 - Modo: “criptografar” ou “descriptografar”.
- **Processamento:**
 - Tratar letras maiúsculas e minúsculas (converter para maiúsculas internamente).
 - Espaços e pontuação devem ser mantidos no resultado SEM cifrar.
 - A chave deve ser repetida ciclicamente apenas sobre as letras (ignorar espaços).
 - Validar que a chave contém apenas letras (erro se conter números ou símbolos).
 - Aplicação da fórmula: $C[i] = (M[i] + K[i]) \bmod 26$ para criptografar e $M[i] = (C[i] - K[i] + 26) \bmod 26$ para descriptografar.
- **Output:** Mensagem cifrada/decifrada, exibição da chave expandida, deslocamentos aplicados e visualização do **Quadrado de Vigenère** na interface.

Missão 3: Escada de Assalto (Cálculo de Inclinação Máxima por Peso do Soldado)

- **Objetivo:** Implementar um CALCULADOR DE INCLINAÇÃO DE ESCADA que determine, com base no peso do soldado (em kg), a inclinação máxima segura da escada (em graus), e vice-versa.
 - **Input:**
 - Peso do soldado (kg): número decimal (ex: 95.5).
 - Peso do equipamento (kg): número decimal (ex: 15.0).
 - Modo: “peso → ângulo” ou “ângulo → peso”.
 - Se modo “ângulo → peso”: informar o ângulo desejado (graus).
 - **Processamento:**
 - Calcular sempre o peso TOTAL (Soldado + Equipamento).
 - Fórmula Ângulo Máximo (graus): $75 - (\text{Peso_Total_kg} - 80) \times 0.15$.
 - Fórmula Peso Máximo (kg): $80 + (75 - \text{Ângulo_Desejado}) / 0.15$.
 - Limitar o ângulo entre 50 e 85 graus (exibir aviso se fora do intervalo).
 - Exibir os cálculos intermediários (peso total, fórmula aplicada, resultado).
 - Validar entradas: peso deve ser número positivo maior que zero.
 - Tratar casos de borda: peso muito baixo, ângulo fora do intervalo.
 - **Output:** Resultado numérico, Classificação de Risco (**SEGURO** ($\leq 70^\circ$), **ATENÇÃO** (71° - 78°), **CRÍTICO** ($>78^\circ$)) e exibição dos cálculos passo a passo.
-

6. Sistema de Avaliação e Critérios

A avaliação será conduzida por uma banca composta por **3 a 5 jurados** (professores e profissionais do mercado).

Critério	Peso	Descrição
Qualidade Técnica	40%	Estabilidade do código, tratamento de erros, precisão dos algoritmos e uso de boas práticas de programação.
Funcionalidade	20%	Cumprimento integral dos requisitos das missões propostas e ausência de bugs críticos.
Interface (UI/UX)	20%	Intuitividade, responsividade, clareza na apresentação dos resultados e bônus para acessibilidade digital.
Documentação	10%	Organização do repositório, histórico de commits claro, uso adequado de branches e clareza do <code>README.md</code> com instruções de instalação e execução.
Apresentação	10%	Capacidade de defesa da solução, agilidade na demonstração “Live Demo” e clareza nas explicações técnicas.

- **Bônus:** Implementação de acessibilidade (Bônus Supremo) e organização exemplar de branches (Bônus Compreensivo).
- **Penalidades:** Falha na execução da demo, atrasos na entrega, problemas para iniciar a aplicação ou plágio resultarão em perda de pontos ou desclassificação imediata.

7. Cronograma de Apresentações e “Live Demo”

As apresentações serão puramente técnicas e focadas na demonstração funcional do código. **O uso de slides (PowerPoint/Canva) é terminantemente proibido.**

7.1. Requisitos da Apresentação

1. **Ambiente Limpo:** O líder da equipe deve iniciar a apresentação com o terminal aberto e sem o código do projeto baixado localmente.
2. **Fluxo Obrigatório de Inicialização:**
 - `git clone [URL_DO_REPOSITARIO_DA_EQUIPE]`
 - `cd [PASTA_DA_MISSAO_ESPECIFICA]`
 - Instalação de dependências (se aplicável): `npm install` (para JS/TS) ou `pip install -r requirements.txt` (para Python).
 - Execução do comando de inicialização da aplicação (ex: `npm start`, `python app.py`, `java -jar app.jar`).

3. **Demonstração Interativa:** A equipe deverá inserir inputs fornecidos pelos jurados em tempo real e validar o output gerado pela aplicação para cada missão.
4. **Foco no Código:** Durante a demonstração, trechos relevantes do código-fonte podem ser exibidos e explicados para a banca, destacando a lógica e as boas práticas.

7.2. Exemplo de Boa Apresentação (Roteiro Sugerido - 10 minutos)

- **Minuto 1:** Clone do repositório, breve explicação da stack tecnológica escolhida (ex: “Usamos Python com Flask para a interface web e a biblioteca `cryptography` para os algoritmos”).
- **Minuto 2-3:** Demonstração da **Missão 1 (RSA)**. O líder executa a aplicação, insere os parâmetros (p, q, e, mensagem) e mostra o texto cifrado/decifrado, explicando brevemente a lógica de cálculo da chave privada `d` no código.
- **Minuto 4-5:** Demonstração da **Missão 2 (Cifra de Vigenère)**. A equipe mostra a aplicação em funcionamento, inserindo mensagem e chave, e destaca como os espaços são tratados e o **Quadrado de Vigenère** é visualizado na interface.
- **Minuto 6-8:** Demonstração da **Missão 3 (Escada de Assalto)**. A equipe testa a aplicação com um peso que resulte em um ângulo crítico, mostrando o alerta visual de risco e explicando a fórmula de cálculo e validação de limites.
- **Minuto 9-10:** Sessão de perguntas e respostas técnicas da banca sobre o código, arquitetura e decisões de implementação.

7.3. Exemplo de Código Esperado (Clean Code)

```
# Exemplo de implementação limpa para a Missão 3 - Escada de Assalto

def calcular_risco(angulo: float) → str:
    """Classifica o risco com base no ângulo de inclinação."""
    if angulo ≤ 70.0:
        return "SEGURO"
    elif 70.0 < angulo ≤ 78.0:
        return "ATENÇÃO"
    else:
        return "CRÍTICO"

def calcular_inclinacao_maxima(peso_total_kg: float) → dict:
    """Calcula a inclinação máxima segura da escada dado o peso total."""
    if peso_total_kg ≤ 0:
        raise ValueError("0 peso total deve ser um número positivo.")

    angulo_max = 75 - (peso_total_kg - 80) * 0.15

    # Limitar o ângulo entre 50 e 85 graus
    angulo_max = max(50.0, min(85.0, angulo_max))

    return {
        "peso_total": peso_total_kg,
        "angulo_max": round(angulo_max, 2),
        "status_risco": calcular_risco(angulo_max)
    }

def calcular_peso_maximo(angulo_desejado: float) → dict:
    """Calcula o peso máximo permitido dado um ângulo de inclinação."""
    if not (50.0 ≤ angulo_desejado ≤ 85.0):
        raise ValueError("0 ângulo desejado deve estar entre 50 e 85 graus")

    peso_max = 80 + (75 - angulo_desejado) / 0.15

    return {
        "angulo_desejado": angulo_desejado,
        "peso_maximo": round(peso_max, 2),
        "status_risco": calcular_risco(angulo_desejado)
    }

# Exemplo de uso (para demonstração)
```



```
# resultado_angulo = calcular_inclinacao_maxima(peso_soldado=90, peso_equi:  
# print(resultado_angulo)  
# resultado_peso = calcular_peso_maximo(angulo_desejado=72)  
# print(resultado_peso)
```

8. Checklist da Organização (Logística e Infraestrutura)

Para o sucesso da operação, os seguintes itens devem ser validados e garantidos pela organização:

- ☐ **Infraestrutura de Rede:** Wi-Fi de alta velocidade e pontos de rede cabeada estáveis no Laboratório 205, com capacidade para múltiplos acessos simultâneos.
- ☐ **Ambiente Técnico:** Imagem das VMs testada e replicada em todos os terminais, garantindo que todas as dependências listadas na Seção 3.1 estejam pré-instaladas e funcionando corretamente.
- ☐ **Suporte Técnico:** Um técnico de TI qualificado deve estar de plantão e acessível durante todo o período noturno do evento para resolver quaisquer imprevistos técnicos relacionados à infraestrutura.
- ☐ **Energia:** Disponibilidade de estabilizadores, extensões e régua de tomada adicionais para que os participantes possam carregar notebooks pessoais e outros dispositivos.
- ☐ **Comunicação:** Sistema de som, microfones e projetores em perfeito funcionamento para a cerimônia de abertura, briefings e apresentações finais.
- ☐ **Alimentação e Hidratação:** Cronograma de Coffee Breaks e pontos de hidratação (água, café) constantes para manter os participantes energizados.
- ☐ **Contingência:** Backup físico dos arquivos das missões em pendrives ou em um servidor local acessível, para o caso de falhas na rede ou no acesso ao QR Code.

9. Checklist de Entrega Final dos Projetos

Para que o projeto de uma equipe seja considerado apto para avaliação, os seguintes requisitos devem ser rigorosamente cumpridos:

- ☐ **Repositório GitHub:** O código-fonte de todas as missões deve estar hospedado em um repositório GitHub (público ou com acesso concedido à banca avaliadora).
- ☐ **Organização de Branches:** Cada missão deve ser desenvolvida e entregue em uma branch separada (ex: `mission-1`, `mission-2`, `mission-3`).

- ☐ **Arquivo README.md** : Um arquivo README.md detalhado deve ser fornecido na raiz de cada branch de missão, contendo:
 - Instruções claras e passo a passo para instalação de dependências e execução da aplicação.
 - Breve descrição da solução implementada.
 - Exemplos de inputs e outputs esperados.
 - ☐ **Código Dinâmico**: O código não deve conter “hardcoded values” para inputs críticos; deve aceitar entradas dinâmicas, seja via interface gráfica, linha de comando ou API.
 - ☐ **Funcionalidade**: Todas as funcionalidades exigidas para cada missão devem estar implementadas e operacionais.
-

10. Estratégia de Divulgação e Marketing

Para garantir a máxima participação e engajamento, as seguintes estratégias de divulgação serão implementadas:

- **Canais Internos**: Utilização de e-mail institucional, portal do aluno e grupos oficiais de WhatsApp/Telegram da UNIMETROCAMP para alcançar todos os estudantes de Computação.
 - **Visual Impactante**: Criação de cartazes digitais e físicos com temática “Cyber-Ops” e contagem regressiva, exibidos nos monitores da unidade e em murais.
 - **Presencial**: Divulgação ativa em sala de aula pelos coordenadores de curso e representantes de turma, com sessões de tira-dúvidas sobre o evento.
 - **Engajamento Digital**: Publicação de “Teasers” com pequenos enigmas e desafios nas redes sociais oficiais da faculdade, gerando curiosidade e interação pré-evento.
 - **Parcerias Internas**: Colaboração com outros departamentos e centros acadêmicos para ampliar o alcance da divulgação.
-

11. Melhorias Futuras e Expansão

Para as próximas edições da “Operation: A Cobra Vai Fumar”, planeja-se a evolução do evento com as seguintes possibilidades:

- **Abertura Externa**: Convidar estudantes de outras instituições de ensino da região para participar, promovendo a integração e o intercâmbio de conhecimentos.
- **Parcerias Corporativas**: Buscar patrocínio de empresas de tecnologia para oferecer premiações mais robustas, oportunidades de estágio/emprego e realizar um “Job Fair”

durante o evento.

- **Categorias Avançadas:** Inclusão de trilhas de desafios mais complexas, como CyberSecurity (Capture The Flag - CTF), Inteligência Artificial (Machine Learning Challenges) e Blockchain.
- **Incubação de Projetos:** Criar um programa de continuidade para os projetos de destaque, transformando-os em projetos de extensão universitária, pesquisas acadêmicas ou até mesmo startups, com mentoria e suporte da instituição.
- **Internacionalização:** Explorar a possibilidade de parcerias com universidades estrangeiras para futuras edições.

Comissão Organizadora *Hackathon UNIMETROCAMP Wyden “A Cobra Vai Fumar!”*